# Boletín U11 — Inicial

## 📝 Boletín U11 — Inicial

Para los ejercicios prácticos necesitas MongoDB funcionando en `localhost:27017` (puedes usar MongoDB Community instalado en tu máquina o Atlas en la nube) y, para los de Java, el driver `mongodb-driver-sync` añadido al proyecto. Los ejercicios marcados con 🧪 usan db4o y son **opcionales**: solo puedes hacerlos si consigues el `.jar`.

Usaremos una colección `peliculas` en una base de datos `cine`, con documentos como este:

```
{ "titulo": "Alien", "anio": 1979, "genero": "terror", "nota": 8.5 }
```

1. Explica con tus palabras qué es el **desfase objeto-relacional** y cómo lo resuelve una base de datos orientada a objetos. Pon un ejemplo con una clase `Cliente` que tiene una lista de `Pedido`.

2. Di si cada afirmación es verdadera o falsa y corrige las falsas:

   - a) En una BDOO hace falta definir tablas antes de guardar objetos.
   - b) Una BDOO puede guardar objetos que contienen a otros objetos.
   - c) db4o es un servidor de bases de datos que hay que instalar aparte.
   - d) En MongoDB todos los documentos de una colección deben tener exactamente los mismos campos.

3. Clasifica cada caso en la familia NoSQL que mejor encaja (documentos, clave-valor, columnas anchas o grafos) y justifica brevemente: **(a)** guardar la sesión de los usuarios conectados a una web; **(b)** el catálogo de productos de una tienda, donde cada producto tiene atributos distintos; **(c)** las amistades entre usuarios de una red social; **(d)** miles de millones de lecturas de sensores.

4. Rellena la tabla de equivalencias entre los conceptos relacionales y los de MongoDB: tabla, fila, columna, clave primaria, clave ajena.

5. En el shell de MongoDB (`mongosh`), inserta en la colección `peliculas` de la base `cine` cinco películas con `insertOne` / `insertMany`, con al menos dos géneros distintos.

6. Escribe, en el shell, las consultas para: **(a)** mostrar todas las películas; **(b)** las de un género concreto; **(c)** las posteriores al año 2000; **(d)** las de nota igual o superior a 8, ordenadas de mejor a peor nota; **(e)** solo las tres primeras.

7. Escribe, en el shell, las sentencias para: **(a)** cambiar la nota de una película; **(b)** sumar 0,5 a la nota de todas las películas de un género; **(c)** borrar una película por su título.

8. Escribe un programa Java que se conecte a MongoDB, obtenga la colección `peliculas` e **inserte** una película con `insertOne`, usando `try-with-resources` para el `MongoClient`.

9. Escribe un programa Java que **muestre por pantalla** el título y el año de todas las películas de la colección, una por línea.

10. Explica la diferencia entre **embeber** y **referenciar** documentos en MongoDB. Para un `autor` con varios `libros`, ¿cuál usarías si los libros solo se consultan junto con su autor? ¿Y si un libro puede tener varios autores?

11. 🧪 Con la clase `Piloto` de los apuntes: almacena tres pilotos en un fichero db4o, recupéralos todos con una consulta por ejemplo (QBE) y muéstralos. Recuerda cerrar la base de datos.

12. 🧪 Con la misma base de datos, recupera un piloto, súmale 10 puntos y guarda el cambio. Comprueba, abriendo de nuevo la base, que sigue habiendo **tres** pilotos y no cuatro. ¿Qué habría pasado si hubieras creado un `new Piloto(...)` con los datos nuevos en vez de modificar el recuperado?